

Conversational QoR Debugging Agent for Open ASIC Flows

Keywords: LLM, QoR Debugging, Flow Automation, EDA Copilot

1. Introduction

This project implements a conversational assistant that parses synthesis and place-and-route reports from open ASIC flows and explains Quality-of-Results issues in natural language. The system combines LibreLane/OpenROAD automation, deterministic report parsing, rule-based diagnosis logic, and an optional LLM-backed explanation layer.

The deliverables are:

- A working debugging assistant with rule-based and optional LLM modes, accessible from the command line.
- Automated evaluation covering 24 predefined troubleshooting scenarios including adversarial tests.
- A test suite with 110 unit and integration tests. Run `python scripts/validate_project.py` to verify.
- A usability report documenting strengths and limitations.

2. PM32 Benchmark and LibreLane Setup

2.1 Design Description

The benchmark is PM32, a 32-bit signed serial-parallel multiplier. It uses carry-save addition internally.

The top-level module (`pm32.v`) is an FSM with three states: IDLE, RUNNING, and DONE. It processes one bit per cycle, requiring 64 clock cycles for a 32x32 multiplication. The datapath (`spm.v`) implements a chain of carry-save adders and a two's complement module.

This is an architecture-level trade-off. A tree multiplier (Wallace or Dadda) would complete in roughly one cycle but needs quadratic area and a long critical path. The serial-parallel approach uses linear area but linear cycles. Our QoR agent reveals the physical cost of this choice: area around 15,943 square micrometers, power around 1.35 mW, and setup slack of 13.237 ns at a 25 ns clock period.

The carry-save adder chain is a likely timing contributor. Using the 25 ns target and 13.237 ns margin, an approximate path delay near 11.76 ns can be inferred, though detailed path reports would be needed to confirm.

2.2 Experimental Setup

Ten clock configurations were explored for the PM32 clock-period sweep. Complete report artifacts are available for `clock_11`, `clock_12`, `clock_25`, and `clock_30`. Two aggressive

targets (clock_8 and clock_10) were executed but the flow terminated before producing final metrics. Four intermediate targets (clock_15 through clock_22) remain config-only.

Run ID	T_clock (ns)	Target Freq (MHz)	Status
clock_8	8	125.0	execution_failed
clock_10	10	100.0	execution_failed
clock_11	11	90.9	completed
clock_12	12	83.3	completed
clock_15	15	66.7	config_only
clock_18	18	55.6	config_only
clock_20	20	50.0	config_only
clock_22	22	45.5	config_only
clock_25	25	40.0	completed
clock_30	30	33.3	completed

Notes:

- config_only means the configuration file exists but no execution was attempted.
- execution_failed means LibreLane ran but could not converge at that clock target.
- completed means the full flow finished and reports are preserved.

The four completed runs span the timing boundary from borderline (1.237 ns margin at 11 ns) to very relaxed (17.237 ns margin at 30 ns). LibreLane v3.0.3 was used for all executed runs.

2.3 LibreLane/OpenROAD Flow

The ASIC implementation flow proceeds as follows:

1. Synthesis (Yosys) converts RTL to a gate-level netlist using sky130 standard cells.
2. Floorplanning (OpenROAD) defines the die and core area.
3. Placement (OpenROAD) positions cells to minimize wirelength.
4. Clock Tree Synthesis distributes the clock with minimal skew.
5. Routing (OpenROAD) connects cells with metal wires satisfying design rules.
6. Static Timing Analysis verifies all paths meet setup and hold constraints.
7. Report generation produces metrics.json with timing, area, power, and violations.

LibreLane wraps this flow with configuration-driven automation. Our parser extracts QoR metrics from step 7.

3. System Architecture

The system is organized as a pipeline:

PM32 RTL and config files feed into run_experiments.py, which generates qor_runs.csv metadata and optionally executes LibreLane. LibreLane produces metrics JSON and summary RPT files. The parser (parse_existing_runs.py) extracts these into a normalized qor_metrics.csv and parsed_summary.json. The rule engine (rule_engine.py) analyzes the parsed data and produces structured diagnosis objects. The assistant (assistant.py) routes

user questions to appropriate handlers and formats natural-language answers. Optionally, `llm_client.py` can enhance explanations using an external LLM.

4. Report Parsing Methodology

4.1 Input Formats

The parser handles two OpenROAD output formats: structured JSON metrics with standardized key names like `timing__setup__wns` and `timing__setup__ws`, and tabular summary reports in human-readable format.

4.2 Metric Extraction Logic

OpenROAD reports two distinct timing metrics. WNS (Worst Negative Slack) is zero when no violation exists and negative when timing fails. WS (Worst Slack) is the positive margin.

The parser stores both separately and computes a normalized `setup_slack`:

- If WNS is negative, `setup_slack` equals WNS (timing failure).
- If WNS is zero or positive and WS is available, `setup_slack` equals WS (margin).
- If only WNS is available, `setup_slack` equals WNS.

For RPT files, regex extraction identifies the Overall row after validating the header. The RPT Worst Slack column reports WS (positive margin), not WNS.

4.3 Multi-Corner Timing

The parser extracts corner-specific timing values from OpenROAD JSON, computes normalized per-corner slack, and identifies the worst setup and hold corners across all available corners.

4.4 Run Discovery and Mapping

Runs are discovered primarily from `qor_runs.csv` metadata. An explicit mapping file (`run_mapping.json`) connects report filenames to run IDs. A fallback token-based matching system handles generic filenames.

4.5 Missing Data Policy

Fields that cannot be extracted are left empty. A confidence rating (high, medium, low, none) indicates evidence quality for each run.

5. Structured Metric Schema

The normalized CSV output contains 37 fields per run, including:

- `run_id`, `design`, `clock_period`
- `setup_wns`, `setup_ws`, `setup_tns` (raw OpenROAD values)

- hold_wns, hold_ws, hold_tns
- setup_slack, hold_slack (normalized values used for diagnosis)
- area, utilization
- power_total, power_internal, power_switching, power_leakage
- slew_violations, cap_violations, fanout_violations
- route_drc_errors, route_wirelength, route_vias
- grt_wirelength, grt_vias, antenna_violations
- congestion_overflow, congestion_status
- worst_setup_corner, worst_hold_corner, timing_corners_count
- decision, source_files, field_sources, missing_fields, confidence, parser_warnings

6. Rule-Based Diagnosis Logic

The rule engine produces structured diagnosis objects with severity, category, finding, evidence, and recommendation. All timing decisions use normalized setup_slack and hold_slack, not raw WNS.

Configurable thresholds:

- Large setup margin: 5.0 ns
- Critical setup margin: 0.5 ns
- High utilization: 80%
- Very high utilization: 85%

The recommendation priority hierarchy (from highest to lowest priority):

1. Missing or invalid report data
2. Final routing DRC errors
3. Hold timing failure
4. Setup timing failure
5. Slew violations
6. Capacitance violations
7. Fanout violations
8. Explicit congestion overflow
9. Very high utilization
10. Very small positive setup margin
11. Reasonable setup margin
12. Large setup margin, otherwise clean
13. Clock-period optimization (only when sufficiently clean)

This means that if a run has DRC errors, hold failure, and setup failure simultaneously, the assistant correctly identifies DRC as the primary issue to fix first.

Example diagnosis object:

```
run_id: clock_25
severity: info
category: setup_timing
finding: Setup timing met with large margin
evidence: setup_wns = 0, setup_ws = 13.237, setup_slack = 13.237
```

recommendation: Try a smaller CLOCK_PERIOD in the next experiment.

7. Conversational Assistant Design

The assistant routes natural-language questions to specialized handlers using score-based intent matching. Every response cites metric values and source files. Missing data triggers explicit "not available" messages.

It supports both one-shot mode (command-line arguments) and an interactive REPL. A structured data interface (answer_structured) returns intent, data, and text for programmatic use by tests and evaluation.

The assistant distinguishes between execution-failed runs (where LibreLane was attempted but could not converge) and config-only runs (where no execution was attempted). It provides appropriate guidance for each case.

Multi-turn conversation state tracks the selected run, previously compared runs, and last intent, allowing pronoun resolution for follow-up questions like "the first one" or "that run."

The system supports 14+ command types covering summary, fastest setup-clean run, timing diagnosis, tuning recommendation, QoR status with Pareto analysis, what-to-fix priority, metrics display, violations, power, area, congestion, comparison, per-run explanation, missing data, and help.

8. Optional LLM Explanation Layer

The LLM layer is strictly optional. It activates with the --llm flag or the USE_LLM=true environment variable. It falls back to rule-based mode if no API key is available.

The LLM receives only structured metrics and diagnoses, never raw logs. A two-level validation system checks responses: token-level verification against an allowlist of known values, and structured claim validation that checks metric-value pairs against the parsed data. This catches fabricated claims like "power is 25 W" even when 25 appears elsewhere as a clock period.

9. Evaluation Scenarios and Results

9.1 Evaluation Philosophy

The evaluation is scenario-based, not a statistical ML benchmark. A QoR debugging tool has a small set of high-stakes behaviors that must all be correct: it must never fabricate timing data, it must select the correct fastest setup-clean run, and it must clearly distinguish available from unavailable metrics.

9.2 What the Evaluator Checks

The evaluator combines structured fact checks with keyword and forbidden-text checks. Structured checks validate facts such as which run is selected as fastest, whether a global conclusion is possible, which runs are missing data, and whether congestion is quantifiable.

Keyword checks verify that responses contain terms proving real data is reported. Forbidden hallucination checks ensure responses do not contain fabricated metrics.

9.3 Results

All 24 scenarios pass: 10 core commands, 8 natural language variants, 3 adversarial/hallucination safety tests, and 3 terminology checks.

This confirms that the assistant uses real parsed metrics, does not fabricate missing data, correctly distinguishes WNS from WS, selects the correct fastest run, provides evidence-based recommendations, and reports routing data honestly.

9.4 Limitations of the Evaluation

The scenario set is manually crafted and does not test adversarial inputs exhaustively. Multi-turn evaluation is not supported (each scenario is independent). The evaluation does not test numerical precision in all output contexts. These limitations are acceptable for a deterministic CLI assistant with controlled input vocabulary.

10. Auto-Tuning

The project includes an iterative clock period auto-tuning script that implements a loop: generate a config, optionally run LibreLane, discover generated reports, update metadata, invoke the parser, read the actual setup_slack, decide the next clock period, and record the iteration history.

Safeguards include minimum and maximum clock bounds, no repeated periods, maximum iteration count, and stopping when reports are unavailable. Dry-run mode generates configs and shows proposed execution without modifying canonical metadata.

11. Limitations

1. Four out of ten planned configurations have complete reports. Two failed during execution. Four remain config-only. A global best cannot yet be determined.
2. Detailed routing completed with zero final DRC errors, but explicit congestion overflow data is unavailable. Zero DRC does not prove zero congestion.
3. Power values are from a single activity estimate.
4. The diagnosis engine does not perform detailed path-group reasoning or full cross-corner path analysis.
5. Only one benchmark (PM32).
6. The LLM layer depends on external API availability.

7. Live auto-tuning depends on LibreLane availability.
8. Historical LibreLane/OpenROAD versions were not recorded for the original clock_25 and clock_30 report artifacts.

13. Conclusion

This project demonstrates that the gap between architecture-level design decisions and physical implementation quality can be bridged through systematic parsing, rule-based diagnosis, and conversational explanation.

The technical contribution includes deterministic parsing with correct WNS/WS distinction, rule-based diagnosis with configurable thresholds and a 13-level priority hierarchy, a conversational interface making physical implementation feedback accessible, an optional LLM layer that adds fluency without compromising reliability, and multi-corner timing extraction identifying worst corners across process/voltage/temperature conditions.

The evaluation confirms that all 24 predefined troubleshooting scenarios pass, including adversarial hallucination-safety tests. This demonstrates scenario coverage, not statistical generalization. The deterministic-first approach to QoR explanation is both reliable and useful.

Understanding physical QoR is not merely a back-end concern. It is essential feedback for architecture-level decision-making.